PrimeTek Africa – Codecamp Academy

Capstone Project Technical Guidelines and Development Roadmap

Advanced Full-Stack Software Development Program

Project Phase: Capstone Projects

Item	Detail
Organization	PrimeTek Africa – Codecamp Academy
Program	Advanced Full-Stack Software Development
Module	Capstone Projects
Instructor	Cristian Camilo Cortes Ortiz
Role	Systems Engineer and Full-Stack Software Development Instructor
Project phase	Capstone Projects

Table of Contents

1. Introduction
- 1.1 Purpose of this document;
- 1.2 Intended audience;
- 1.3 How to read this document;
- 1.4 Classification of content;
- 1.5 Terminology

2. Program and Capstone Context
- 2.1 Institutional information;
- 2.2 Role of the Capstone phase;
- 2.3 Prerequisite knowledge

3. Capstone Objectives
- 3.1 General objective;
- 3.2 Specific objectives;
- 3.3 Mandatory requirements and recommendations

4. Curriculum and Technical Knowledge
- 4.1 Software Engineering Fundamentals;
- 4.2 Version Control and Collaborative Development;
- 4.3 Frontend Development;
- 4.4 Backend Development;
- 4.5 Database Engineering;
- 4.6 Software Testing and Quality Assurance;
- 4.7 Containerization and Infrastructure;
- 4.8 DevOps and Continuous Integration;
- 4.9 Cybersecurity and Secure Development

5. Project Definition and Requirements
- 5.1 Required sections of a project definition;
- 5.2 Functional and non-functional requirements;
- 5.3 Examples of well-written requirements;
- 5.4 Business rules, scope, and acceptance criteria

6. Software Architecture and Technical Design
- 6.1 Architecture documentation;
- 6.2 Diagrams and when to use them;
- 6.3 Architecture Decision Records

7. Recommended Technology Stack
- 7.1 Reference architecture;
- 7.2 Technology selection guidelines;
- 7.3 Reference diagram

8. Development Methodology
- 8.1 Agile workflow;
- 8.2 User stories;
- 8.3 Definition of Done;
- 8.4 Organizing work in GitHub;
- 8.5 Individual and collaborative development

9. Repository and Collaboration Standards
- 9.1 Repository structure;

9.2 Monorepo and separate repositories;
9.3 .gitignore and .env.example;
9.4 README standard;
9.5 Contribution workflow
10. Project Implementation Roadmap
10.1 Phases 1 to 9;
10.2 Adapting the roadmap
11. Testing and Quality Assurance
11.1 Testing strategy;
11.2 REST API tests;
11.3 Authentication tests;
11.4 GraphQL tests;
11.5 Real-time tests;
11.6 Testing throughout the lifecycle
12. Cybersecurity Requirements
12.1 OWASP Top 10 as a reference;
12.2 Minimum security controls;
12.3 Identifying, preventing, and documenting risks
13. DevOps and Deployment
13.1 Containerizing the full-stack application;
13.2 Basic CI/CD pipeline;
13.3 Deployment and production validation
14. Technical Documentation Standards
14.1 Documentation set;
14.2 API documentation;
14.3 Database documentation
15. Deliverables and Evaluation Framework
15.1 Deliverables checklist;
15.2 Evaluation framework
16. Final Presentation Requirements
17. Official Technical References
18. Appendices and Templates
Appendix A to Appendix I

1. Introduction

1.1 Purpose of this document

This document is the primary technical reference for the Capstone Project phase of the Advanced Full-Stack Software Development program at PrimeTek Africa – Codecamp Academy. It states the technical expectations, development standards, and implementation roadmap that students follow from the first project proposal to the final presentation.
The document does three things. It consolidates the knowledge developed during the program so that students can see how each topic contributes to a complete application. It defines how a Capstone project is described, designed, built, tested, secured, deployed, and documented. It gives students a phased plan that can be adapted to the size of the project and the sprint structure set by the instructor.

1.2 Intended audience

The audience is students who have completed the foundational and technical stages of the program and are moving into professional full-stack development. The instructor uses the document to plan supervision and evaluation. Reviewers can use it to understand what a project is expected to demonstrate.

1.3 How to read this document

Sections 1 to 3 give the context and objectives. Section 4 summarizes the curriculum and explains how each domain affects a Capstone project. Sections 5 to 10 define how to specify, design, plan, and organize the work. Sections 11 to 14 cover quality, security, delivery, and documentation. Sections 15 and 16 describe deliverables, evaluation, and the final presentation. Sections 17 and 18 contain references and templates.

1.4 Classification of content

To keep the document accurate, every technology or practice is classified into one of three categories. Students must be able to tell the categories apart.

Label	Meaning	Student obligation
Program Topic	A topic covered during the Advanced Full-Stack Software Development program.	Students may use it freely and are expected to understand it.
Recommended Practice	A professional engineering practice that supports quality, security, or maintainability. It may use tools that were not part of the program.	Strongly encouraged. Where it is not mandatory, the student explains any decision not to follow it.

Label	Meaning	Student obligation
Optional Technology	An additional technology that students may introduce into a project.	Allowed when it is justified in an Architecture Decision Record (Section 6.3) and does not replace a required deliverable.

Unless a statement says otherwise, the rules in this document are recommendations. A requirement that is mandatory is written with the word "must" and is listed in Section 3.3. Mandatory status for any item that is not already a program topic requires instructor confirmation.

1.5 Terminology

Term	Definition used in this document
Capstone project	The final practical project that integrates the knowledge acquired in the program.
Requirement	A statement of something the system must do (functional) or a quality it must have (non-functional).
Acceptance criterion	A verifiable condition that must be true for a requirement or user story to be accepted.
Definition of Done (DoD)	A shared checklist that every work item satisfies before it is considered complete.
Architecture Decision Record (ADR)	A short document that records a technical decision, its context, the alternatives, and the consequences.
API	Application Programming Interface exposed by the backend, through REST, GraphQL, or both.
CI/CD	Continuous Integration and Continuous Delivery or Deployment.
RBAC	Role-Based Access Control.
ORM	Object-Relational Mapping, here Sequelize for PostgreSQL.

2. Program and Capstone Context

2.1 Institutional information

Field	Value
Organization	PrimeTek Africa – Codecamp Academy
Program	Advanced Full-Stack Software Development
Module	Programming Foundations
Instructor	Cristian Camilo Cortes Ortiz
Role	Systems Engineer and Full-Stack Software Development Instructor
Project phase	Capstone Projects

2.2 Role of the Capstone phase

The Capstone phase is the practical integration of the knowledge acquired throughout the program. During earlier stages students studied individual technologies in isolation: a language feature, a framework, a database, a tool. A Capstone project requires them to combine these pieces into one application with a defined purpose, a defined set of users, a documented design, automated checks, and a deployment procedure. The value of the phase lies in the decisions that connect the technologies: how data flows between layers, how errors are handled, how access is controlled, and how the team keeps the code in a releasable state.

2.3 Prerequisite knowledge

Students are expected to arrive with the knowledge summarized in Section 4. Gaps discovered during planning should be reported to the instructor early, so that they can be addressed through targeted practice instead of surfacing late in the project.

3. Capstone Objectives

3.1 General objective

Design, build, test, secure, containerize, document, and present a complete full-stack software application that solves a clearly defined problem, applying the technical knowledge acquired in the Advanced Full-Stack Software Development program and following professional engineering standards. *Proposed wording; requires instructor confirmation.*

3.2 Specific objectives

1. Define a real or realistic problem, its justification, users, scope, and requirements in a written project definition.
2. Produce architecture and database designs, supported by diagrams, before and during implementation.
3. Implement a backend API with Node.js and Express.js, using REST, GraphQL, or both, with authentication and authorization where the project requires them.
4. Implement a frontend application with React or Vue.js, styled with Bootstrap or Tailwind CSS, that consumes the API.
5. Persist data in PostgreSQL through Sequelize and, where the project justifies it, in MongoDB.
6. Apply automated testing, secure development practices, and the OWASP Top 10 as a security reference.
7. Package the application with Docker and Docker Compose and automate checks with GitHub Actions.
8. Work with Git and GitHub using an Agile-oriented workflow, and communicate the result through documentation and a final presentation.

3.3 Mandatory requirements and recommendations

The following statements are proposed as mandatory for every Capstone project. They follow directly from the purpose of the phase. They require instructor confirmation before they apply as official requirements.

- A project must have a written definition that includes the fifteen sections listed in Section 5.1.
- A project must be developed in a Git repository hosted on GitHub, with a documented history of contributions.
- A project must contain a backend, a database, and a user-facing client, unless the instructor approves a different scope.
- A project must not commit credentials, tokens, private keys, or other secrets to the repository.
- A project must include a README that allows another person to install and run the application.
- Technology choices must be justified in writing. A project is not required to use every technology in this document.

Everything else in this document is a recommendation unless the instructor states otherwise.

4. Curriculum and Technical Knowledge

This section organizes the program into nine domains. Each domain lists the concepts covered in the program, explains how they influence a professional project, and points to the sections where they are applied. The Classification column uses the labels defined in Section 1.4.

4.1 Software Engineering Fundamentals

Concept group	Topics	Classification
Programming basics	Variables, data types, operators, control structures, loops, functions, modular programming.	Program Topic
Object-oriented programming	Classes, objects, inheritance, encapsulation, abstraction, polymorphism.	Program Topic
Execution models	Event-driven programming, asynchronous programming.	Program Topic
Reliability	Error handling and debugging.	Program Topic
Design quality	Clean code principles, separation of concerns, code reusability and maintainability.	Program Topic
Process and structure	Software development lifecycle, basic software architecture principles.	Program Topic

How these concepts shape a professional project. Functions and modules determine how a codebase is divided. A project whose logic lives in a few large files cannot be reviewed, tested, or shared among team members. Modular programming and separation of concerns lead to a layered backend in which routes only describe endpoints, controllers translate HTTP requests into operations, services contain business rules, and models handle persistence. Each layer can be tested independently and changed without rewriting the others.

Object-oriented concepts appear in domain models and in the way related behavior is grouped. Encapsulation keeps invariants, such as a booking that cannot end before it begins, inside one place instead of scattering the check across the application. Inheritance should be used sparingly. Composition and small functions are usually easier to maintain than deep class hierarchies.

Event-driven and asynchronous programming are central to JavaScript and Node.js. Every database query, network request, and file operation is asynchronous, so students must handle promises and errors consistently. Unhandled promise rejections cause failures that are hard to trace. A project should adopt one error-handling convention and apply it everywhere.
Clean code principles reduce the cost of change: descriptive names, functions that do one thing, no duplicated logic, and comments that explain intent instead of restating code. The software development lifecycle, from requirements to maintenance, is the frame for Section 10. Basic architecture principles, such as layering and low coupling, are developed in Section 6.

4.2 Version Control and Collaborative Development

Concept group	Topics	Classification
Git fundamentals	Repository management, commits, commit conventions, remote repositories.	Program Topic
Integration	Branching strategies, merging, rebasing, conflict resolution.	Program Topic
GitHub collaboration	Pull requests, code reviews, issues, milestones, GitHub project management, collaborative workflows.	Program Topic
Commit message format	Conventional Commits specification.	Recommended Practice

Recommended Git workflow for a team. A Capstone team shares one repository. The workflow below, often called GitHub Flow, is simple enough for a small team and compatible with continuous integration.

1. The main branch always contains working code. Direct pushes to it are disabled through branch protection (Section 13.2).
2. Each task starts from an issue. The developer creates a short-lived branch from the latest main, named after the work, for example feature/42-booking-endpoint or fix/57-token-expiry.
3. The developer commits in small steps with messages that follow Conventional Commits, for example feat(auth): add refresh token endpoint.
4. The developer opens a pull request that references the issue and describes what changed, how it was tested, and anything reviewers should look at.
5. At least one teammate reviews the pull request. Automated checks must pass.
6. The pull request is merged into main. The branch is deleted.

A team that needs a staging area for integration can add a long-lived develop branch and merge it into main at the end of each sprint. This adds overhead, so it should be used only when the team can explain why it needs it.

Merging and rebasing. Rebasing a personal feature branch onto the latest main keeps the history linear and surfaces conflicts early. Rebasing a branch that other people have pulled rewrites shared history and must be avoided. A merge commit preserves the exact history and is safe on shared branches.

Conflict resolution. Conflicts are resolved by the author of the branch being integrated, by reading both versions, deciding what the combined behavior should be, and running the tests before committing the resolution. Frequent integration and small pull requests keep conflicts

small.

Code reviews. A reviewer checks correctness against the acceptance criteria, readability, test coverage, security concerns, and consistency with the architecture. Review comments address the code, not the author. The author responds to every comment.

4.3 Frontend Development

HTML, CSS, and CSS frameworks

Area	Topics	Classification
HTML	Semantic HTML, document structure, forms and input elements, accessibility fundamentals, HTML validation.	Program Topic
CSS	Selectors and specificity, box model, Flexbox, CSS Grid, positioning, responsive design, media queries, CSS variables, animations and transitions, mobile-first development.	Program Topic
CSS frameworks	Bootstrap, Tailwind CSS, component-based styling, responsive utility classes, design consistency.	Program Topic

Semantic HTML and accessibility are the basis of a usable interface. Using the correct elements (button, nav, label, headings in order) gives keyboard and screen reader support without extra code. Forms must have labels, and validation messages must be readable by assistive technology. A mobile-first approach means the base styles target small screens and media queries add layout for larger ones, which matches how most users will reach a deployed project.

Bootstrap and Tailwind CSS solve the same problem in different ways. Bootstrap provides ready-made components and a grid. Tailwind CSS provides utility classes composed in markup. A project should choose one, apply it consistently, and avoid mixing both in the same interface.

JavaScript

Area	Topics	Classification
Language	ES6+ features, variables and scope, functions and arrow	Program Topic

Area	Topics	Classification
	functions, arrays and objects, destructuring and spread operators, array methods, modules, error handling, JSON processing.	
Browser	DOM manipulation, event handling.	Program Topic
Asynchronous work	Promises, async/await, Fetch API.	Program Topic

In a Capstone project JavaScript is the common language across frontend and backend. Consistent use of modules, const and let, array methods such as map and filter, and async/await with try/catch makes the code uniform and readable.

React

Area	Topics	Classification
React	Component-based architecture, functional components, props and state, hooks (useState, useEffect), component lifecycle concepts, event handling, conditional rendering, lists and keys, React Router, API integration, frontend application structure.	Program Topic

Vue.js

Area	Topics	Classification
Vue.js	Vue fundamentals, Single File Components, Composition API, ref and reactive, computed properties, watch, component communication, Vue Router, API integration, frontend application structure.	Program Topic

Choosing between React and Vue

Both frameworks build the interface from components that render from state, and both can consume the same REST or GraphQL API. A project uses one of them. Students are not required to use both frameworks in a single project.

Aspect	React	Vue.js
Component format	JavaScript functions that return JSX.	Single File Components with template, script, and style sections.
Reactivity	State updates through hooks such as useState; the component function runs again on change.	Reactive state with ref and reactive; templates update when tracked values change.
Derived data	Calculated during rendering, or memoized when needed.	Computed properties with automatic dependency tracking.
Side effects	useEffect.	watch, watchEffect, and lifecycle hooks.
Routing	React Router.	Vue Router.
Typical learning path	Requires comfort with JavaScript and JSX.	Templates resemble HTML, which some students find easier at the start.

Selection guidelines. Choose the framework the team understands best, because delivery risk matters more than framework features in a time-limited project. If team members have different strengths, choose the framework in which the weakest member can still contribute. Consider how much of the interface is dynamic: both frameworks handle it, so the project's requirements rarely decide the matter. Record the choice and the reasons in an ADR. Do not switch frameworks after Phase 5 begins.

4.4 Backend Development

Area	Topics	Classification
Node.js	Runtime fundamentals, modules and package management, npm, asynchronous execution, event loop fundamentals, environment variables, file and directory management,	Program Topic

Area	Topics	Classification
	error handling.	
Express.js	Application structure, routing, controllers, middleware, request and response handling, input validation, centralized error handling, REST API development, API versioning, HTTP status codes.	Program Topic
API architecture	REST principles, HTTP methods, resource-based architecture, JSON communication, request validation, API documentation, pagination and filtering, API versioning, GraphQL fundamentals, schemas and types, queries and mutations, resolvers, Apollo Server, GraphQL context, REST versus GraphQL.	Program Topic
Authentication and authorization	Authentication fundamentals, authorization concepts, JWT, password hashing, login and registration workflows, RBAC, protected routes, API key validation, authorization middleware.	Program Topic
Real-time communication	WebSockets fundamentals, Socket.IO, client-server communication, events and listeners, real-time messaging, rooms, connection lifecycle, error handling and reconnection.	Program Topic
Security middleware	helmet, express-rate-limit, a schema validation library.	Recommended Practice / Optional Technology (library choice)

Layered Express structure. A backend that follows separation of concerns uses the following layers. A request passes through global middleware, then a route, then a controller, then a service, then a model, and the response or an error travels back through the same chain.

```
src/
  app.js              Express app: middleware, routes, error handlers
  server.js           HTTP server start (and Socket.IO attachment)
  config/             Environment-based configuration
  routes/             Route definitions per resource, versioned under
/api/v1
  controllers/        HTTP-level logic: read input, call service, send
response
  services/           Business rules, transactions
  models/             Sequelize models and associations
  middleware/         Authentication, authorization, validation,
errors
  graphql/            typeDefs and resolvers (when GraphQL is used)
  sockets/            Socket.IO event handlers (when real time is
used)
  migrations/         Database migrations
  seeders/            Database seed data
```

Application setup with centralized error handling. Every unexpected error reaches one handler that decides what to return. Internal details are logged and never sent to the client. In Express 5, a rejected promise inside an async route handler is passed to the error handler automatically. In Express 4, async handlers must call next(error) themselves or be wrapped.

```
// src/middleware/errors.js
export class HttpError extends Error {
  constructor(status, message, details) {
    super(message);
    this.status = status;
    this.details = details;
  }
}

export const notFound = (req, res, next) =>
  next(new HttpError(404, 'Resource not found'));

export const errorHandler = (err, req, res, next) => {
  const status = err.status ?? 500;
  if (status >= 500) console.error(err);   // log internally
  res.status(status).json({
    error: {
      message: status >= 500 ? 'Internal server error' : err.message,
      details: status >= 500 ? undefined : err.details,
    },
  });
};
// src/app.js
import express from 'express';
```

```
import cors from 'cors';
import helmet from 'helmet';
import rateLimit from 'express-rate-limit';
import routes from './routes/index.js';
import { notFound, errorHandler } from './middleware/errors.js';

const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.CORS_ORIGIN?.split(',') }));
app.use(express.json({ limit: '100kb' }));
app.use(rateLimit({ windowMs: 15 * 60 * 1000, limit: 100,
standardHeaders: true }));

app.use('/api/v1', routes);   // API versioning through the URL prefix
app.use(notFound);
app.use(errorHandler);

export default app;
```

The example uses ES modules ("type": "module" in package.json). The packages helmet, cors, and express-rate-limit are Recommended Practice tools. cors must be configured with an explicit list of allowed origins in production.

HTTP status codes. Use 200 for successful reads and updates, 201 for created resources, 204 for successful requests with no body, 400 for malformed or invalid input, 401 when authentication is missing or invalid, 403 when the user is authenticated but not permitted, 404 when a resource does not exist, 409 for conflicts such as a duplicate unique value, 422 where the team prefers it for semantic validation errors, and 500 for unexpected server errors. The team picks one convention for validation errors and uses it everywhere.

REST principles. Resources are nouns identified by URLs (/api/v1/bookings/15). HTTP methods express the action: GET reads, POST creates, PUT replaces, PATCH changes part of a resource, DELETE removes. GET must not change state. Collections support pagination (?page=2&limit=20) and filtering (?status=confirmed), and responses use consistent JSON shapes.

GraphQL fundamentals. A GraphQL API exposes one endpoint and a typed schema. Clients ask for exactly the fields they need through queries, and change data through mutations. Resolvers are the functions that produce each field. The context object is created for every request and typically carries the authenticated user and data access objects.

```
// graphql/server.js
import { ApolloServer } from '@apollo/server';
import { startStandaloneServer } from '@apollo/server/standalone';
import { GraphQLError } from 'graphql';

const typeDefs = `#graphql
  type Task { id: ID! title: String! done: Boolean! }
```

```
  type Query { tasks: [Task!]! }
  type Mutation { createTask(title: String!): Task! }
`;

const resolvers = {
  Query: { tasks: (_, __, ctx) => ctx.models.Task.findAll() },
  Mutation: {
    createTask: (_, { title }, ctx) => {
      if (!ctx.user) {
        throw new GraphQLError('Authentication required', {
          extensions: { code: 'UNAUTHENTICATED' },
        });
      }
      return ctx.models.Task.create({ title });
    },
  },
};

const server = new ApolloServer({ typeDefs, resolvers });
const { url } = await startStandaloneServer(server, {
  listen: { port: 4000 },
  context: async ({ req }) => ({ user: getUserFromToken(req), models
}),
});
```

The standalone server keeps the example short. To run GraphQL inside an existing Express application, follow the Apollo Server documentation for the installed major version, because the Express integration package differs between versions.

Criterion	REST	GraphQL
Endpoints	Many, one per resource.	One, with a typed schema.
Response shape	Defined by the server for each endpoint.	Defined by the client for each query.
Over-fetching and under-fetching	Common; may require several requests.	Reduced; one query can return related data.
Caching	Straightforward with HTTP caching.	Needs additional design.
Learning cost	Lower.	Higher: schema, resolvers, and query design.
Good fit	Standard CRUD resources, simple clients.	Clients that need flexible, nested data from several resources.

A project may use REST, GraphQL, or both. Using both needs a reason, for example REST for

authentication and file operations and GraphQL for queries. A single approach is usually enough.

Authentication and authorization. Authentication establishes who the user is. Authorization decides what that user may do. Passwords are never stored in plain text. They are hashed with a slow, salted algorithm such as bcrypt. After login, the server issues a JSON Web Token (JWT) signed with a secret kept in an environment variable. Protected routes verify the token on every request, and authorization middleware compares the role in the token with the roles the route allows (RBAC). API key validation applies where a client system, not a person, calls the API.

```
// src/middleware/auth.js
import jwt from 'jsonwebtoken';
import { HttpError } from './errors.js';

export function authenticate(req, res, next) {
  const [scheme, token] = (req.headers.authorization ?? '').split('
');
  if (scheme !== 'Bearer' || !token) {
    return next(new HttpError(401, 'Authentication required'));
  }
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);  // { sub,
role, ... }
    next();
  } catch {
    next(new HttpError(401, 'Invalid or expired token'));
  }
}

export const authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user?.role)
    ? next()
    : next(new HttpError(403, 'Insufficient permissions'));

// Usage: router.delete('/rooms/:id', authenticate,
authorize('admin'), removeRoom);
```

Tokens should have a short expiration time. If the project uses refresh tokens, they must be stored and revoked on the server side. See Section 12 for secure token handling.

Real-time communication. WebSockets keep a connection open so that the server can send data to clients without waiting for a request. Socket.IO builds on WebSockets and adds named events, rooms, acknowledgements, and automatic reconnection. A real-time feature, such as a chat between staff and guests, needs the same authentication and authorization checks as the REST API.

```
// src/sockets/index.js
```

```
import { Server } from 'socket.io';
import jwt from 'jsonwebtoken';

export function attachSockets(httpServer) {
  const io = new Server(httpServer, {
    cors: { origin: process.env.CORS_ORIGIN?.split(',') },
  });

  io.use((socket, next) => {                    // authenticate the
connection
    try {
      socket.data.user = jwt.verify(socket.handshake.auth.token,
process.env.JWT_SECRET);
      next();
    } catch {
      next(new Error('unauthorized'));
    }
  });

  io.on('connection', (socket) => {
    socket.on('room:join', (roomId) => {
      // Verify here that the user is allowed to join this room.
      socket.join(`room:${roomId}`);
    });
    socket.on('message:send', ({ roomId, text }) => {
      io.to(`room:${roomId}`).emit('message:new', {
        from: socket.data.user.sub,
        text,
        sentAt: new Date().toISOString(),
      });
    });
    socket.on('disconnect', (reason) => console.log('disconnected:',
reason));
  });
  return io;
}

// Client: const socket = io(API_URL, { auth: { token } });
// socket.on('connect_error', ...) handles rejected connections.
// Socket.IO reconnects automatically after network interruptions.
```

4.5 Database Engineering

Area	Topics	Classification
Relational databases	Tables, records, relationships, primary and foreign keys, one-to-one, one-to-many, and many-to-many relationships, SQL fundamentals, CRUD, JOIN operations, constraints, indexes, normalization, transactions, referential integrity, migrations, seeding. PostgreSQL is the primary relational technology.	Program Topic
ORM	Sequelize: models and associations, database configuration, migrations, seeders, query operations, transactions, data validation.	Program Topic
Non-relational databases	NoSQL fundamentals, document-oriented databases, collections and documents, schema flexibility, CRUD, data modeling, MongoDB, MongoDB Atlas, application integration.	Program Topic

Relational design. Start from the entities and their relationships, draw an Entity Relationship Diagram, and then translate it into tables. Each table has a primary key. A foreign key enforces a relationship and protects referential integrity. Many-to-many relationships need a join table. Normalization removes duplicated data: each fact is stored once. Constraints (NOT NULL, UNIQUE, CHECK, foreign keys) place rules in the database so that no code path can violate them. Indexes speed up reads on columns used in lookups and joins, at the cost of slower writes, so they are added for known query patterns, not by default. Transactions group several changes so that they all succeed or all fail, for example creating an order and reducing stock.

```
// models/booking.js (Sequelize v6 style)
import { DataTypes } from 'sequelize';

export default (sequelize) => {
  const Booking = sequelize.define('Booking', {
    id: { type: DataTypes.UUID, defaultValue: DataTypes.UUIDV4,
primaryKey: true },
    checkIn: { type: DataTypes.DATEONLY, allowNull: false },
    checkOut: { type: DataTypes.DATEONLY, allowNull: false },
```

```
    status: {
      type: DataTypes.ENUM('pending', 'confirmed', 'cancelled'),
      allowNull: false,
      defaultValue: 'pending',
    },
  }, {
    validate: {
      checkOutAfterCheckIn() {
        if (this.checkOut <= this.checkIn) throw new Error('checkOut
must be after checkIn');
      },
    },
  });

  Booking.associate = (models) => {
    Booking.belongsTo(models.User, { foreignKey: 'userId' });     //
one-to-many
    Booking.belongsToMany(models.Service, { through: 'BookingServices'
}); // many-to-many
  };
  return Booking;
};

// Transaction example
await sequelize.transaction(async (t) => {
  const booking = await Booking.create(data, { transaction: t });
  await room.update({ available: false }, { transaction: t });
});
```

Schema changes are made through migrations, never by editing the database by hand. Migrations are versioned in the repository, so every team member and every environment reaches the same schema. Seeders load repeatable test or initial data. Production seed data must not contain real credentials.

Non-relational databases. MongoDB stores documents in collections. The schema is flexible, and related data is often embedded in the same document. MongoDB Atlas is the hosted service. Data modeling in MongoDB follows how the application reads data: data read together is stored together.

Criterion	SQL (PostgreSQL)	NoSQL (MongoDB)
Data model	Tables with fixed columns and relationships.	Documents in collections with a flexible structure.
Integrity	Strong: constraints, foreign keys, transactions.	Enforced mostly in application code or with schema validation.

Criterion	SQL (PostgreSQL)	NoSQL (MongoDB)
Relationships	First-class, queried with JOIN.	Embedded or referenced; no joins by default.
Schema changes	Through migrations.	Easier to vary, but the application must handle old and new shapes.
Typical fit	Transactional data with clear relationships: users, orders, bookings, payments.	Variable or nested data: logs, content, catalogs with differing attributes, chat history.

Selecting a database architecture. Use PostgreSQL as the default for a Capstone project. Add MongoDB only when part of the data is clearly document-shaped or highly variable, and write the reason in an ADR. Using two databases increases setup, testing, and deployment work, so it must be worth the cost.

SQL injection protection with Sequelize. Use model methods or bind parameters. Never build a query by concatenating user input.

```
// Safe: bound parameters
await sequelize.query(
  'SELECT * FROM "Users" WHERE email = :email',
  { replacements: { email: req.body.email }, type: QueryTypes.SELECT }
);
// Unsafe: `SELECT * FROM "Users" WHERE email = '${req.body.email}'`
```

4.6 Software Testing and Quality Assurance

Area	Topics	Classification
Fundamentals	Testing fundamentals, strategies, assertions, test coverage, error handling validation, regression testing, manual and automated testing.	Program Topic
Levels	Unit, integration, API, and end-to-end testing.	Program Topic
Environments and data	Test environments, test data management.	Program Topic
Tools	Node.js built-in test runner, Jest, Supertest, Playwright, Vitest.	Optional Technology (the program does not prescribe a tool; requires instructor confirmation)

Testing is covered in detail in Section 11. In brief, tests are written during development so that regressions are caught as soon as they appear, and the automated tests run in the CI pipeline on every pull request.

4.7 Containerization and Infrastructure

Area	Topics	Classification
Docker	Containerization fundamentals, images and containers, Dockerfiles, Docker Compose, environment configuration, container networking, persistent volumes, container logs, multi-container applications, database containers, application containerization, development and production configurations.	Program Topic

A container packages an application with its runtime and dependencies so that it behaves the same on every machine. An image is the read-only template, and a container is a running instance. A Dockerfile describes how to build an image. Docker Compose describes several containers, their networks, and their volumes in one file. In a Compose network, containers reach each other by service name, so the backend connects to the database at the host name db, not localhost. Data that must survive container removal, such as the PostgreSQL data directory, is stored in a named volume. Logs are read with docker compose logs. The full-stack procedure is in Section 13.1.

4.8 DevOps and Continuous Integration

Area	Topics	Classification
DevOps practice	DevOps fundamentals, development and deployment workflows, CI, continuous delivery, continuous deployment, build automation, deployment automation.	Program Topic
GitHub Actions	Workflow configuration, YAML fundamentals, automated testing pipelines, environment variables and secrets, branch protection, pull request	Program Topic

Area	Topics	Classification
	validation, repository automation.	

Continuous Integration merges and verifies changes frequently, running automated checks for each change. Continuous Delivery keeps the application in a state that can be released at any time, with a manual approval for release. Continuous Deployment releases every change that passes the checks automatically. A Capstone project is expected to implement CI and, where a hosting target exists, a basic delivery step. See Section 13.2.

4.9 Cybersecurity and Secure Development

Area	Topics	Classification
Principles	Secure software development principles, principle of least privilege, error handling without exposing sensitive information.	Program Topic
Input and output	Input validation and sanitization, SQL injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF).	Program Topic
Identity and access	Authentication security, authorization and access control, password hashing, secure session and token management.	Program Topic
Platform	CORS configuration, rate limiting, API security, secure HTTP communication, environment variable protection, secrets management, dependency security.	Program Topic
Reference	OWASP Top 10.	Program Topic

Security is applied across the whole project and not as a final step. Section 12 defines the minimum controls and the process for documenting risks.

5. Project Definition and Requirements

5.1 Required sections of a project definition

Every Capstone project begins with a written definition that contains the following sections. The definition is the first deliverable and is approved by the instructor before implementation begins.

No.	Section	Content
1	Project title	A short, descriptive name.
2	Project overview	One or two paragraphs: what the system is and what it does.
3	Problem statement	The problem, who has it, and its current consequences.
4	Problem justification	Why the problem deserves a software solution, with evidence where available.
5	General objective	One sentence stating the overall goal of the system.
6	Specific objectives	Measurable steps that together achieve the general objective.
7	Target users	The types of people who use the system and their goals.
8	Stakeholders	Everyone affected by or interested in the system, including people who do not use it directly.
9	Functional requirements	What the system does.
10	Non-functional requirements	Qualities the system has: performance, security, usability, reliability, maintainability.
11	Business rules	Policies and constraints of the domain that the system enforces.
12	Project scope	What is included in this version.

No.	Section	Content
13	Project limitations	What is excluded, and constraints of time, technology, or data.
14	Expected outcomes	The deliverables and results the project will produce.
15	Acceptance criteria	Conditions that show the project meets its objectives.

5.2 Functional and non-functional requirements

A functional requirement describes a behavior: something the system does in response to input or events. A non-functional requirement describes a quality or constraint: how well the system does it, or under what conditions. Functional requirements answer "what"; non-functional requirements answer "how well". Both must be verifiable. A requirement that cannot be tested, such as "the system must be fast", is rewritten with a measurable condition.

Aspect	Functional	Non-functional
Concerns	Features and behavior.	Quality attributes and constraints.
Typical wording	"The system shall allow..."	"The system shall ... within / at least / no more than..."
Verified by	Functional and acceptance tests.	Performance checks, security review, usability review, code inspection.
Affects	Endpoints, screens, business logic.	Architecture, technology choices, infrastructure.

5.3 Examples of well-written requirements

The following examples use a hotel booking system as an illustration. Numeric thresholds in non-functional requirements are examples; each project sets values appropriate to its context.

ID	Requirement	Why it is well written
FR-01	The system shall allow a registered guest to create a booking by selecting a room	Names the actor, the action, and the inputs.

ID	Requirement	Why it is well written
	type, a check-in date, and a check-out date.	
FR-02	The system shall reject a booking when the check-out date is not later than the check-in date and shall return an error message that identifies the invalid field.	States a verifiable condition and a verifiable response.
FR-03	The system shall allow a user with the staff role to view all bookings for a selected date.	Ties a capability to a role, which supports RBAC.
NFR-01	Passwords shall be stored only as salted hashes produced by a slow hashing algorithm.	Testable by inspecting the database and the code.
NFR-02	At least 95 percent of API read requests shall complete in under 500 ms in the test environment with 50 concurrent users.	Measurable with a defined load and environment.
NFR-03	The interface shall be usable on screen widths from 360 px to 1920 px without horizontal scrolling.	Verifiable by inspection at stated widths.
NFR-04	The application shall start with one command using Docker Compose on a machine that has Docker installed.	Verifiable by following the README.

Weak wording to avoid: "user-friendly", "fast", "secure", "as needed", and "etc." Each of these hides an unstated condition.

5.4 Business rules, scope, and acceptance criteria

Business rules are statements about the domain that exist independently of the software, such as "A booking cannot be cancelled less than 24 hours before check-in." The system enforces them, preferably in the service layer and, where possible, with database constraints. **Scope** lists what is in the first release. **Limitations** lists what is deliberately excluded. Writing limitations protects the team from uncontrolled growth in size. **Acceptance criteria** are written so that a person who did not build the system can check them. For a user story they can follow the

format: Given a context, when an action occurs, then an observable result follows.

6. Software Architecture and Technical Design

6.1 Architecture documentation

Students produce architecture documentation before implementation begins and update it as the design changes. The documentation does not need to be long. It must be accurate and sufficient for a new team member to understand how the system is organized.

Architecture view	What it describes
System architecture	The main parts of the system, their responsibilities, and how they communicate, including external services.
Frontend architecture	Framework, folder structure, routing, state management, API client, and component organization.
Backend architecture	Layers, modules, middleware, error handling, configuration.
Database architecture	Database engine or engines, schema, relationships, indexes, migration strategy.
API architecture	REST resources and versioning and/or GraphQL schema, error format, pagination, documentation approach.
Authentication architecture	Registration and login flow, password hashing, token issue and verification, roles and permissions, protected routes.
Communication architecture	Request/response flows, real-time channels, events, rooms, and reconnection behavior.
Infrastructure architecture	Containers, networks, volumes, environments, CI/CD pipeline, and hosting target.

6.2 Diagrams and when to use them

Diagram	Use it when	What it must communicate
Use Case Diagram	Defining scope and user goals during Phase 1.	Actors, their goals (use cases), and the system boundary.
Entity Relationship Diagram (ERD)	Designing the relational database, in Phase 2.	Entities, attributes, keys, relationship types, and cardinality.
Class Diagram	Designing domain objects and service classes.	Classes, attributes, methods, and relationships such as association and inheritance.
Component Diagram	Showing how the application is divided into parts.	Components, their interfaces, and dependencies (for example frontend, API, database, external service).
Sequence Diagram	Explaining an interaction in time, such as login or a booking request.	Participants, message order, and responses, including error paths.
Activity Diagram	Describing a workflow or business process with decisions.	Steps, decisions, parallel paths, and end states.
Deployment Diagram	Documenting where the software runs.	Nodes such as containers or servers, the artifacts on them, and the network links.

Diagrams may be drawn with any tool the team prefers. Text-based diagram languages such as Mermaid render in GitHub Markdown files and are a convenient Optional Technology because the diagram source is versioned with the code. A diagram that is not kept current is worse than no diagram, so each diagram has an owner and is updated when the design changes.

6.3 Architecture Decision Records

Students must document important architectural decisions and justify their technical choices. A decision is important when it is expensive to reverse or affects several parts of the system: choice of frontend framework, REST or GraphQL, one or two databases, authentication approach, and deployment target. Each decision is recorded in a short ADR stored in the repository (for example in docs/adr/) using the template in Appendix C. The justification should refer to the project's requirements and to the team's skills, not to popularity.

7. Recommended Technology Stack

7.1 Reference architecture

The architecture below is a reference. It is not a mandatory combination of every technology. Students select technologies based on project requirements and justify their decisions.

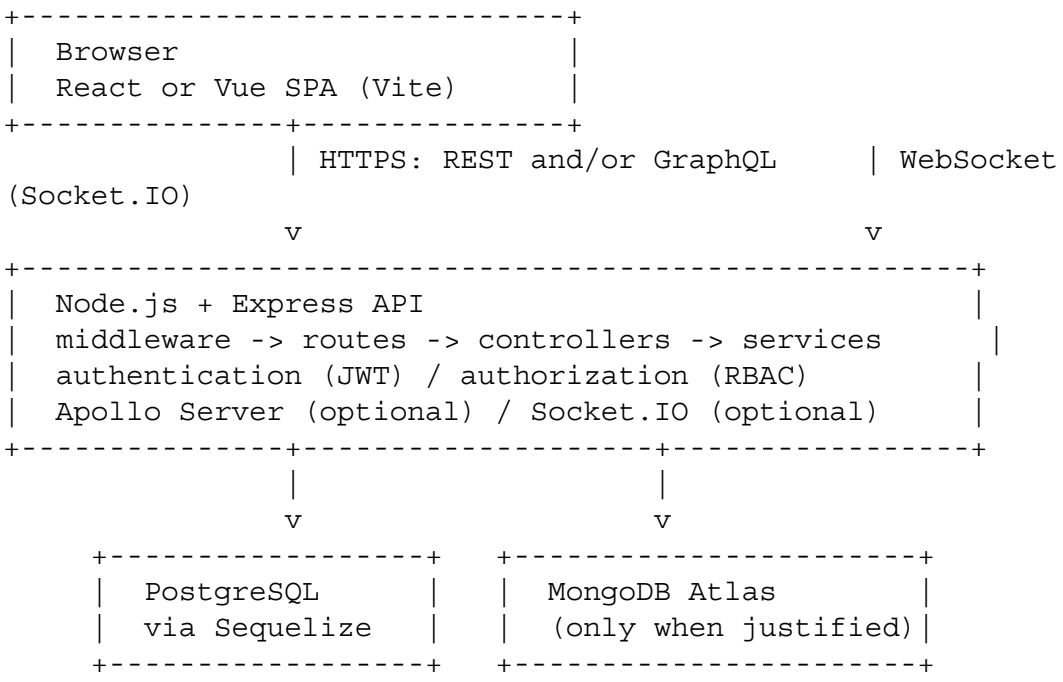
Layer	Reference technologies	Notes
Frontend	React or Vue; Vite; React Router or Vue Router; Bootstrap or Tailwind CSS.	Choose one framework and one CSS approach.
Backend	Node.js; Express.js; REST API and/or GraphQL; authentication and authorization; Socket.IO when real-time communication is required.	Add GraphQL and Socket.IO only when the requirements call for them.
Database	PostgreSQL with Sequelize; MongoDB with MongoDB Atlas when appropriate.	PostgreSQL is the default; MongoDB needs a recorded reason.
Infrastructure	Docker; Docker Compose; environment configuration.	One Compose file should start the whole system.
DevOps	GitHub; GitHub Actions; CI/CD pipelines.	CI is the minimum; deployment automation depends on the hosting target.
Testing	Unit, integration, API, and end-to-end testing.	Tools are chosen by the team; see Section 11.
Security	OWASP security principles; secure authentication; input validation; access control.	Applies to every layer.

7.2 Technology selection guidelines

1. Start from the requirements. List what the system must do and which qualities matter most.
2. Prefer technologies from the program. Each additional technology needs time for learning, setup, testing, and documentation.
3. Consider the team's skills and the schedule. A familiar technology used well is better than

an unfamiliar one used poorly.
4. Prefer technologies with official documentation and active maintenance.
5. Avoid unnecessary complexity and over-engineering. Do not add GraphQL, a second database, or real-time features without a requirement that needs them.
6. Record each significant decision in an ADR.

7.3 Reference diagram



Development: Docker Compose runs frontend, backend, and database.
Delivery: GitHub repository -> GitHub Actions (test, build) -> deployment.

8. Development Methodology

8.1 Agile workflow

Capstone projects follow an Agile-oriented workflow. The goal is to deliver working software in small increments, to inspect the result regularly, and to adapt the plan based on what is learned. The structure below can be scaled down for an individual project.

Element	Description
Project planning	Based on the project definition, the team identifies epics and estimates the effort at a coarse level.

Element	Description
Product backlog	An ordered list of everything that may be built: user stories, technical tasks, and defects. The most valuable and least certain items are placed first.
Epics, user stories, and tasks	An epic is a large feature area. It is divided into user stories that deliver value to a user. Each story is divided into tasks that one person can complete in a short time.
Sprint planning	The team chooses the backlog items it commits to for the sprint and defines the sprint goal. Sprint length is set by the instructor (requires instructor confirmation).
Sprint execution	The team works on the selected items and keeps the main branch releasable.
Daily progress tracking	A short daily check-in, which may be asynchronous: what was done, what is next, and what blocks progress. The GitHub Project board is updated to match.
Sprint review	The team demonstrates the working increment to the instructor and gathers feedback.
Sprint retrospective	The team discusses what worked, what did not, and chooses one or two improvements for the next sprint.
Incremental development	Each sprint produces a usable slice that includes backend, frontend, database changes, and tests, not a single layer in isolation.

8.2 User stories

A user story uses this structure: "As a [type of user], I want [functionality] so that [expected benefit]." Each story has acceptance criteria.

Story	Acceptance criteria
As a guest, I want to register with my email and a password so that I can manage my bookings.	Given a new email, when I submit valid data, then my account is created and I receive a success response. Given an email already in

Story	Acceptance criteria
	use, when I submit, then I see an error and no second account is created. The password is stored as a hash.
As a staff member, I want to see today's check-ins so that I can prepare rooms.	Given I am logged in with the staff role, when I open the dashboard, then I see all bookings with today's check-in date. A guest cannot access this page.
As a guest, I want to send a message to the front desk so that I can ask for help during my stay.	Given I have an active booking, when I send a message, then staff in the same room receive it in real time. If my connection drops, the client reconnects automatically.

A good story is independent, small enough to finish in one sprint, testable, and written in terms of user value. Technical tasks that do not map to a user story, such as setting up CI, are recorded as tasks under an epic such as "Project setup".

8.3 Definition of Done

A work item is done only when all of the following are true. The team may add items. Changes to this list require instructor confirmation if it is used for evaluation.

- The code meets the acceptance criteria of the story.
- The code follows the agreed code style and has no known defects.
- Automated tests for the new behavior exist and pass.
- The change has been reviewed in a pull request and merged to the main branch.
- CI checks pass.
- Security considerations were reviewed: input validation, access control, and no secrets in the code.
- Relevant documentation was updated: README, API documentation, diagrams.
- The feature works in the Docker Compose environment.

8.4 Organizing work in GitHub

GitHub feature	Recommended use
Issues	One issue per user story or task. The issue contains the story, acceptance criteria, and labels such as feature, bug, documentation, security. Issue templates (Appendix D) keep the format consistent.
Milestones	One milestone per sprint, with a due date. Issues assigned to the milestone show the

GitHub feature	Recommended use
	sprint progress. A milestone can also represent a release.
GitHub Projects	A board with columns such as Backlog, Ready, In Progress, In Review, and Done. A table or roadmap view can show priorities and dates. Fields such as Sprint, Epic, and Assignee help filter the work.
Pull requests	Each pull request links to its issue with a closing keyword such as Closes #12 so that the issue closes when the pull request is merged. The pull request template (Appendix E) lists the checks to complete.
Branch protection	Require pull requests and passing checks before merging to main.

8.5 Individual and collaborative development

Collaborative projects. Assign each story to one owner, with a second person as reviewer. Agree on interfaces early: the API contract between frontend and backend lets both sides work in parallel. Divide work in vertical slices (a feature across all layers) when the team is small, and by layer only when members specialize. Rotate reviewers so that knowledge spreads. Every member must have a visible history of commits and reviews in the repository.
Individual projects. The same practices apply on a smaller scale. Use issues and a board, work on branches, and merge through pull requests, even without a reviewer, so that the history is clear and CI runs on every change. Keep the scope small enough to finish, and prefer a complete, tested core to many incomplete features.

9. Repository and Collaboration Standards

9.1 Repository structure

An example monorepo layout follows. The names can be adapted if the structure stays consistent and is documented in the README.

```
capstone-project/
  .github/
    workflows/ci.yml
    ISSUE_TEMPLATE/
    pull_request_template.md
```

```
backend/
  src/
  tests/
  Dockerfile
  package.json
  .env.example
frontend/
  src/
  public/
  Dockerfile
  package.json
  .env.example
docs/
  architecture.md
  api.md
  database.md
  adr/
  diagrams/
docker-compose.yml
.env.example
.gitignore
README.md
LICENSE (see license considerations)
```

9.2 Monorepo and separate repositories

Approach	Advantages	Appropriate when
Monorepo (one repository)	One place for issues, pull requests, and CI. A change that spans frontend and backend is one pull request. Simpler Docker Compose setup.	The default for Capstone projects, particularly for small teams where the same people work on both parts.
Separate frontend and backend repositories	Independent versioning and deployment. Clear ownership boundaries. Smaller pipelines.	Different teams own each part, the parts are deployed independently, or the instructor requires separate repositories. Requires coordination of the API contract and extra setup effort.

9.3 .gitignore and .env.example

Never commit credentials, tokens, private keys, or other secrets. Configuration that differs between environments is read from environment variables. The repository contains an .env.example file that lists every variable with a safe placeholder value, and the real .env file is excluded by .gitignore. If a secret is committed by mistake, it must be revoked and replaced, because removing it from the latest commit does not remove it from the history.

```
# .gitignore
node_modules/
dist/
build/
coverage/
.env
.env.*
!.env.example
*.log
.DS_Store
.vscode/
.idea/
docker-compose.override.yml   # only if it contains local-only
settings
# .env.example  (placeholders only; copy to .env and fill in real
values)
NODE_ENV=development
PORT=4000
CORS_ORIGIN=http://localhost:5173

DB_HOST=db
DB_PORT=5432
DB_NAME=capstone_db
DB_USER=capstone_user
DB_PASSWORD=change-me

JWT_SECRET=replace-with-a-long-random-value
JWT_EXPIRES_IN=15m

# Optional, only when MongoDB is used
MONGODB_URI=mongodb+srv://user:password@cluster.example.mongodb.net/db
name
```

In GitHub Actions and in hosting platforms, secrets are stored in the platform's secret store and never in the repository.

9.4 README standard

The README is the entry point of the project. It must allow a reader who has not seen the project to understand it and run it. The following sections are expected.

Section	Content
Project description	Purpose, main features, and target users.
Technology stack	Technologies used and a link to the ADRs that justify them.
Installation instructions	Prerequisites with versions (Node.js, Docker) and the exact steps to obtain the code and install dependencies.
Environment configuration	How to create .env from .env.example and what each variable means.
Dependencies	Main runtime dependencies and where they are declared.
Development instructions	How to start the system locally, run migrations and seeders, and use the development scripts.
API documentation	A summary and a link to the full documentation.
Architecture and database documentation	Links to the architecture and database documents and diagrams.
Testing instructions	How to run each type of test.
Deployment instructions	How the application is deployed and how to verify it.
Contribution guidelines	Branching, commit conventions, pull request process, and code style.
License	The license, or a statement that none has been chosen.

License considerations. Without a license, the default is that others have no permission to reuse the code. Students should ask the instructor whether the institution has rules about ownership and licensing of Capstone work before choosing a license (requires instructor confirmation). Every dependency also has its own license, which the team should be aware of.

9.5 Contribution workflow

The contribution workflow in Section 4.2 applies. The CONTRIBUTING.md file summarizes it: how to create a branch, the commit message format, how to run tests and linters before opening a pull request, and what reviewers look for.

10. Project Implementation Roadmap

10.1 Phases 1 to 9

The roadmap divides the project lifecycle into nine phases. Durations are not fixed in this document and are set by the instructor (requires instructor confirmation).

Phase	Activities	Main outputs
Phase 1 – Project Discovery and Definition	Identify the problem. Define project objectives. Identify stakeholders. Define scope. Establish requirements.	Project proposal; requirements specification; use case diagram.
Phase 2 – Research and Technical Planning	Technology selection. Feasibility analysis. Architecture design. Database design. API planning. Security planning.	Architecture documents and diagrams; ERD; API plan; ADRs; initial security risk list.
Phase 3 – Project Setup	Repository creation. Branch strategy. Development environment. Backend initialization. Frontend initialization. Database configuration. Docker configuration.	Repository with structure, README skeleton, project board, working Docker Compose, branch protection.
Phase 4 – Backend Implementation	Database models. Relationships. Migrations and seeders. API development. Authentication. Authorization. Business logic. API testing.	Working API with documented endpoints and automated API tests.
Phase 5 – Frontend Implementation	UI architecture. Component development. Routing. State management. API integration. Authentication workflows. Responsive design. User experience.	Responsive interface that consumes the API, with login flow and protected views.
Phase 6 – Integration	Frontend and backend integration. Database integration. Authentication integration. Real-time functionality where necessary. End-to-end workflows.	Complete user workflows running end to end.

Phase	Activities	Main outputs
Phase 7 – Testing and Security	Unit testing. Integration testing. API testing. Functional testing. Security validation. Bug fixing. Performance considerations.	Test suite, coverage report, security checklist results, list of fixed and known issues.
Phase 8 – DevOps and Deployment	Docker configuration. CI/CD implementation. Automated testing. Environment configuration. Deployment. Production validation.	Working CI pipeline; deployment procedure; verified deployed application.
Phase 9 – Final Documentation and Presentation	Technical documentation. Architecture explanation. Demonstration. Project presentation. Final evaluation.	Final documentation set and presentation (Section 16).

10.2 Adapting the roadmap

The phases can overlap and should be adapted to the project's scope and sprint structure. In practice, testing begins in Phase 4 and not in Phase 7, CI is set up in Phase 3 or as soon as the first tests exist, and security is considered from Phase 2. Backend and frontend work can proceed in parallel once the API contract is defined. Sprints deliver vertical slices, so Phases 4 to 6 are often interleaved. A small project may combine phases, and a larger one may split them. The team should record how it adapted the roadmap in its project plan.

11. Testing and Quality Assurance

11.1 Testing strategy

Level	Purpose	Typical subject
Unit	Verify one function or module in isolation, with dependencies replaced where needed.	Validation functions, business rules in services, utility functions, React or Vue components.
Integration	Verify that several parts work together.	Service plus database; controller plus service.
API	Verify HTTP or GraphQL behavior: status codes, response bodies,	Endpoints and GraphQL operations against a test database.

Level	Purpose	Typical subject
	authentication.	
End-to-end	Verify a complete user workflow in a browser against the running system.	Register, log in, create a booking.
Regression	Confirm that previously working behavior still works.	The full automated suite run in CI.
Manual	Exploratory checks and usability review that automation does not cover.	Visual layout, accessibility checks, unusual workflows.

Tests need a dedicated test environment. They run against a separate test database that is created and reset automatically, never against development or production data. Test data is created by the tests or by seeders designed for testing. Each test sets up what it needs and does not depend on the order of other tests. Assertions state the expected outcome precisely, including the error cases and the response format for errors. Coverage reports show which code the tests execute. They are a guide for finding untested areas and not a goal in themselves. The examples below use the Node.js built-in test runner and Supertest. These tools are an Optional Technology. The team may use Jest, Vitest, or another tool if it is justified, and the concepts are the same.

11.2 REST API tests

```
// tests/health.test.js   (run with: node --test)
import test from 'node:test';
import assert from 'node:assert/strict';
import request from 'supertest';
import app from '../src/app.js';

test('GET /api/v1/health returns status ok', async () => {
  const res = await request(app).get('/api/v1/health');
  assert.equal(res.status, 200);
  assert.equal(res.body.status, 'ok');
});

test('POST /api/v1/bookings rejects invalid dates', async () => {
  const res = await request(app)
    .post('/api/v1/bookings')
    .set('Authorization', `Bearer ${testToken}`)
    .send({ checkIn: '2026-12-10', checkOut: '2026-12-09' });
  assert.equal(res.status, 400);
  assert.ok(res.body.error.message);
});
```

11.3 Authentication tests

```
test('protected route returns 401 without a token', async () => {
  const res = await request(app).get('/api/v1/bookings');
  assert.equal(res.status, 401);
});

test('login returns a token and never returns the password hash',
async () => {
  const res = await request(app)
    .post('/api/v1/auth/login')
    .send({ email: 'guest@example.com', password: 'a-test-password'
});
  assert.equal(res.status, 200);
  assert.ok(res.body.token);
  assert.equal(res.body.user?.passwordHash, undefined);
});

test('guest role cannot access an admin route', async () => {
  const res = await request(app)
    .delete('/api/v1/rooms/1')
    .set('Authorization', `Bearer ${guestToken}`);
  assert.equal(res.status, 403);
});
```

Test accounts use throwaway credentials created for the test, and tokens for tests are generated from a test-only secret defined in the test environment.

11.4 GraphQL tests

```
test('tasks query returns a list', async () => {
  const res = await request(app)
    .post('/graphql')
    .send({ query: '{ tasks { id title done } }' });
  assert.equal(res.status, 200);
  assert.ok(Array.isArray(res.body.data.tasks));
});

test('createTask mutation requires authentication', async () => {
  const res = await request(app)
    .post('/graphql')
    .send({ query: 'mutation { createTask(title: "x") { id } }' });
  assert.equal(res.body.errors[0].extensions.code, 'UNAUTHENTICATED');
});
```

GraphQL servers commonly return HTTP 200 even when an operation fails, so tests inspect the errors array and not only the HTTP status. When the standalone Apollo server is used, tests

start the server on a test port instead of passing an Express app to Supertest.

11.5 Real-time tests

```
import { io as connect } from 'socket.io-client';

test('a user in a room receives messages sent to that room', async ()
=> {
  const a = connect(url, { auth: { token: tokenA } });
  const b = connect(url, { auth: { token: tokenB } });
  await Promise.all([once(a, 'connect'), once(b, 'connect')]);   //
once from node:events

  a.emit('room:join', 'r1');
  b.emit('room:join', 'r1');

  const received = once(b, 'message:new');
  a.emit('message:send', { roomId: 'r1', text: 'hello' });
  const [msg] = await received;
  assert.equal(msg.text, 'hello');

  a.close(); b.close();
});

test('connection without a valid token is rejected', async () => {
  const c = connect(url, { auth: { token: 'invalid' }, reconnection:
false });
  const [err] = await once(c, 'connect_error');
  assert.equal(err.message, 'unauthorized');
  c.close();
});
```

11.6 Testing throughout the lifecycle

Testing is part of development from the first sprint and is not a phase that follows it. Defects found while the code is fresh cost much less to fix than defects found at delivery, when the person who wrote the code may no longer remember the details and many other changes depend on it. Writing tests with each feature also forces the team to turn acceptance criteria into checks, and a suite that runs on every pull request lets the team change code with confidence. Teams that postpone testing to the end usually discover problems when there is no time to fix them. Each user story therefore includes its tests as part of the Definition of Done.

12. Cybersecurity Requirements

12.1 OWASP Top 10 as a reference

The OWASP Top 10 is an awareness document that lists the most critical security risks to web applications. It is a reference for prioritizing, not a complete checklist. The current edition is the OWASP Top 10:2025, which replaced the 2021 edition. Students should consult the official OWASP page for the exact wording and guidance of each category.

OWASP Top 10:2025 category	Typical Capstone control
A01 Broken Access Control	Check authorization on every route and every GraphQL resolver. Verify that users can access only their own resources. Deny by default.
A02 Security Misconfiguration	Restrict CORS origins, set security headers with helmet, disable debug output in production, remove default credentials.
A03 Software Supply Chain Failures	Commit the lock file, install with npm ci, review new dependencies, run npm audit, and keep dependencies updated.
A04 Cryptographic Failures	Hash passwords, use HTTPS, keep secrets out of the code, never invent custom cryptography.
A05 Injection	Use parameterized queries or the ORM. Validate input. Escape output to prevent XSS.
A06 Insecure Design	Identify threats during design (Phase 2) and write security requirements.
A07 Authentication Failures	Rate limit login, enforce password rules, use short-lived tokens, never reveal which part of the credentials was wrong.
A08 Software or Data Integrity Failures	Verify the sources of dependencies and build artifacts. Protect the CI pipeline and its secrets.
A09 Security Logging and Alerting Failures	Log authentication events and access denials without logging passwords or tokens.
A10 Mishandling of Exceptional Conditions	Handle errors centrally, fail safely, return generic messages to clients, and test error paths.

12.2 Minimum security controls

Control	Expectation
Input validation and sanitization	Validate type, length, format, and range of all input on the server, including request bodies, query strings, path parameters, GraphQL arguments, and Socket.IO event payloads. Client-side validation improves usability but is not a security control.
Password hashing	Hash passwords with a slow, salted algorithm such as bcrypt. Never store, log, or return passwords.
Token management	Sign JWTs with a strong secret from the environment, set an expiration, verify on every protected request, and do not store sensitive data in the token payload, which is readable by the client.
Authorization and least privilege	Enforce RBAC on the server. Give each role, database user, and service only the permissions it needs.
SQL injection	Use the ORM or bound parameters. Never concatenate input into queries.
Cross-Site Scripting (XSS)	Rely on the framework's default output escaping in React and Vue. Avoid dangerouslySetInnerHTML and v-html with untrusted content. Consider a Content Security Policy.
Cross-Site Request Forgery (CSRF)	If authentication uses cookies, use SameSite cookie attributes and anti-CSRF tokens. APIs that read a token from the Authorization header are not exposed to CSRF in the same way, but the token storage choice then affects XSS exposure. Document the choice.
CORS	Allow only known origins. Do not combine wildcard origins with credentials.
Rate limiting	Apply limits to all endpoints, with stricter limits on login and registration.
API security	Version the API, validate content types, limit request size, and avoid exposing internal identifiers or stack traces. Apply the same

Control	Expectation
	checks to GraphQL, including limits on query depth where the project is exposed publicly.
Secure HTTP communication	Use HTTPS in any deployed environment. Set security headers.
Secrets and environment variables	Keep secrets out of the repository. Use .env locally, and the platform's secret store in CI and hosting. Use different secrets per environment.
Dependency security	Keep the lock file committed, review dependencies, run npm audit, and update vulnerable packages.
Error handling	Log details internally and return generic messages to clients. Do not expose stack traces, SQL errors, or file paths.

12.3 Identifying, preventing, and documenting risks

1. **Identify.** In Phase 2, list the assets (user data, credentials, bookings), the entry points (endpoints, forms, sockets), and the threats to each. Walk through the OWASP Top 10 categories and note which apply.
2. **Prevent.** For each risk, select a control from Section 12.2 and turn it into a requirement or a task in the backlog.
3. **Verify.** In Phase 7, test the controls: attempt access without a token, with the wrong role, and with another user's resource ID. Submit malformed and hostile input. Run npm audit.
4. **Document.** Maintain a security register in the repository (template in Appendix F) with the risk, its severity, the control, the status, and the evidence. Risks that are accepted or not yet resolved are listed with a reason.
5.

Students test only their own applications and environments.

13. DevOps and Deployment

13.1 Containerizing the full-stack application

A full-stack project runs as three containers: the database, the backend, and the frontend. The files below illustrate the approach. The Node.js image version should be a currently supported LTS release; check the official Node.js release schedule before choosing.

```
# backend/Dockerfile
FROM node:24-alpine
```

```
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
USER node
EXPOSE 4000
CMD ["node", "src/server.js"]
# frontend/Dockerfile  (multi-stage: build with Node, serve static
files with nginx)
FROM node:24-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
# A single-page application with client-side routing needs an nginx
# configuration that falls back to index.html for unknown paths.
# docker-compose.yml
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 5s
      timeout: 5s
      retries: 10

  backend:
    build: ./backend
    environment:
      NODE_ENV: ${NODE_ENV}
      PORT: 4000
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: ${DB_NAME}
      DB_USER: ${DB_USER}
```

```
      DB_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
      CORS_ORIGIN: ${CORS_ORIGIN}
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "4000:4000"

  frontend:
    build:
      context: ./frontend
      args:
        VITE_API_URL: http://localhost:4000/api/v1
    ports:
      - "8080:80"

volumes:
  Db_data:
```

Compose reads the ${...} values from the .env file next to the Compose file, which is excluded from Git. The database is reached from the backend through the service name db. The named volume db_data keeps the data when the container is recreated. The health check ensures that the backend starts only when PostgreSQL accepts connections. Migrations run before the API starts, for example through a start script that executes the Sequelize migration command.
Development and production configurations. A development configuration mounts the source code as a volume and runs the application with a file watcher for fast feedback. A production configuration builds an image without development dependencies, does not mount source code, does not publish the database port, uses secrets from the hosting platform, and runs as a non-root user. A common way to hold both is a base docker-compose.yml and an override file for each environment. Do not copy .env files into images. Add a .dockerignore file that excludes node_modules, .git, and .env.
Useful commands: docker compose up --build, docker compose logs -f backend, docker compose exec db psql -U $DB_USER $DB_NAME, and docker compose down (add -v only to delete the volumes and data).

13.2 Basic CI/CD pipeline

A minimal pipeline runs on every pull request and on every push to the main branch. It installs dependencies, runs linting and tests for the backend against a PostgreSQL service container, and installs, lints, and builds the frontend. Because the database password for CI is created by the workflow environment, it is read from a repository secret, never written into the file.

```
# .github/workflows/ci.yml
name: CI

on:
  pull_request:
```

```
    branches: [main]
  push:
    branches: [main]

jobs:
  backend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    services:
      postgres:
        image: postgres:17
        env:
          POSTGRES_DB: app_test
          POSTGRES_USER: app_test
          POSTGRES_PASSWORD: ${{ secrets.TEST_DB_PASSWORD }}
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U app_test -d app_test"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10
    env:
      NODE_ENV: test
      DB_HOST: localhost
      DB_PORT: 5432
      DB_NAME: app_test
      DB_USER: app_test
      DB_PASSWORD: ${{ secrets.TEST_DB_PASSWORD }}
      JWT_SECRET: ${{ secrets.TEST_JWT_SECRET }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 24
          cache: npm
          cache-dependency-path: backend/package-lock.json
      - run: npm ci
      - run: npm run lint --if-present
      - run: npx sequelize-cli db:migrate
      - run: npm test

  frontend:
    runs-on: ubuntu-latest
    defaults:
      run:
```

```
    working-directory: frontend
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: 24
      cache: npm
      cache-dependency-path: frontend/package-lock.json
  - run: npm ci
  - run: npm run lint --if-present
  - run: npm run build
```

YAML uses indentation to express structure, and indentation errors are a common cause of failing workflows. Secrets are created in the repository settings and are referenced with the secrets context.

Branch protection and pull request validation. In the repository settings, protect the main branch: require a pull request before merging, require at least one approving review, require the CI jobs above to pass, and block force pushes. This makes the pipeline a gate and not a notification.

From CI to CD. Continuous Delivery adds a job that builds the container images and prepares them for release. Continuous Deployment adds a job that deploys automatically after the checks pass on the main branch. The deployment step depends on the hosting target, which the student selects and documents (requires instructor confirmation of acceptable targets). Deployment credentials are stored as secrets and, when the platform supports it, in a protected environment that requires approval.

13.3 Deployment and production validation

1. Prepare production configuration: environment variables, secrets, allowed CORS origin, and HTTPS.
2. Run database migrations in the target environment. Do not run seeders that contain test data in production.
3. Deploy the application using the documented procedure.
4. Validate: open the application, check a health endpoint, register or log in with a test account, exercise a main workflow, and check the logs for errors.
5. Record the deployed URL, the deployment procedure, and known limitations in the README.

14. Technical Documentation Standards

14.1 Documentation set

Document	Location	Content
README	Repository root	See Section 9.4.
Requirements specification	docs/requirements.md	Project definition sections from Section 5.1.
Architecture documentation	docs/architecture.md	The eight views in Section 6.1, with diagrams.
Architecture Decision Records	docs/adr/	One file per decision.
Database documentation	docs/database.md	See Section 14.3.
API documentation	docs/api.md or generated	See Section 14.2.
Security register	docs/security.md	Risks and controls (Appendix F).
Contribution guide	CONTRIBUTING.md	Workflow and standards.

Documentation is written for a reader who joins the project later. It is kept in the repository next to the code and reviewed in pull requests, so that it changes with the code.

14.2 API documentation

For REST, document each endpoint: method and path, purpose, authentication and required role, request parameters and body with types and validation rules, example request, success response with status code, and error responses. For GraphQL, document the schema with descriptions on types and fields, and list example queries and mutations. API documentation can be a Markdown file, an OpenAPI specification, a Postman or similar collection, or a generated interactive page. OpenAPI and interactive documentation tools are Optional Technologies.

```
### POST /api/v1/auth/login
Authenticate a user and return an access token.

Auth: none
Body: { "email": string (valid email), "password": string (min 8) }

200 OK   { "token": string, "user": { "id": string, "email": string,
"role": string } }
400      Validation error
401      Invalid credentials
429      Too many attempts
```

14.3 Database documentation

Include the ERD, a description of each table or collection (purpose, columns or fields, types, constraints), relationships and their cardinality, indexes and the reason for each, the migration and seeding procedure, and the backup or reset procedure for development. Business rules enforced by the database are listed.

15. Deliverables and Evaluation Framework

15.1 Deliverables checklist

This is a proposed checklist. Which items are mandatory, and their due dates, require instructor confirmation. The Status column is for the team's use.

No.	Deliverable	Description	Related phase	Status
1	Project proposal	Title, overview, problem, justification, objectives.	1	
2	Requirements specification	All sections in Section 5.1.	1	
3	Architecture diagrams	The diagrams from Section 6.2 that apply, plus ADRs.	2	
4	Database design	ERD, schema description, migrations.	2, 4	
5	GitHub repository	Structured repository with history, issues, project board, protected main branch.	3	
6	Frontend implementation	Responsive client application.	5	
7	Backend implementation	API with business logic and error	4	

No.	Deliverable	Description	Related phase	Status
		handling.		
8	Database integration	Persistent storage connected to the backend.	4, 6	
9	API documentation	As defined in Section 14.2.	4, 9	
10	Authentication and authorization	Registration, login, protected routes, roles where the project needs them.	4, 6	
11	Automated tests	Unit, integration, and API tests; end-to-end tests where feasible.	4, 7	
12	Docker configuration	Dockerfiles and Compose file that start the system.	3, 8	
13	CI/CD pipeline	GitHub Actions workflow with automated tests.	8	
14	Security considerations	Security register and evidence of controls.	2, 7	
15	Deployment	Deployed application and documented procedure.	8	
16	README documentation	As defined in Section 9.4.	3, 9	
17	Final technical presentation	As defined in Section 16.	9	

15.2 Evaluation framework

The framework below is proposed. No official grading percentages or institutional evaluation

policies are defined in this document. Any weighting, passing threshold, or penalty must be set and approved by the instructor, and the Weighting column is intentionally left to be completed after approval.

Criterion	What is assessed	Evidence	Weighting
Problem definition and project scope	Clarity of the problem, justified objectives, realistic scope, quality of requirements.	Proposal, requirements specification.	Requires instructor approval
Software architecture	Appropriate structure, justified decisions, consistent diagrams.	Architecture documents, ADRs.	Requires instructor approval
Code quality and maintainability	Readability, modularity, separation of concerns, consistent style, absence of duplication.	Source code, pull requests.	Requires instructor approval
Frontend implementation	Component design, routing, state handling, API integration, responsiveness, accessibility, usability.	Running application, source code.	Requires instructor approval
Backend implementation	Layering, business logic, validation, error handling, authentication and authorization.	Running API, source code.	Requires instructor approval
Database design	Correct modeling, relationships, constraints, migrations, seeders, justified database choice.	ERD, migrations, schema.	Requires instructor approval
API design	Consistent resources or schema, status codes, pagination, versioning, documentation.	API documentation, requests.	Requires instructor approval
Security	Implemented controls, handling of secrets,	Security register, code review.	Requires instructor approval

Criterion	What is assessed	Evidence	Weighting
	documented risks.		
Testing	Relevant automated tests, coverage of critical paths and error cases, passing suite.	Test code, CI results.	Requires instructor approval
DevOps and deployment	Working containers, CI pipeline, deployment procedure and validation.	Docker files, workflow, deployed system.	Requires instructor approval
Documentation	Completeness, accuracy, and usefulness of the README and technical documents.	Repository documents.	Requires instructor approval
Collaboration and project management	Use of issues, board, branches, pull requests, reviews, and sprint ceremonies; individual contributions.	GitHub history and board.	Requires instructor approval
Final demonstration	Clarity, working demonstration, ability to explain and defend decisions.	Presentation.	Requires instructor approval

A rubric with four performance levels (for example Exemplary, Proficient, Developing, and Insufficient) with a written descriptor for each criterion is recommended so that expectations are clear before work begins. The descriptors and levels are for the instructor to define.

16. Final Presentation Requirements

The final technical presentation demonstrates the project and explains the engineering decisions behind it. The duration, the format, and whether all team members present are set by the instructor (requires instructor confirmation). The following content is proposed.

Part	Content
1. Problem and users	The problem, the target users, the objectives, and the scope.
2. Architecture	The system architecture, the technology

Part	Content
	choices and the reasons for them, and the database design.
3. Live demonstration	The main user workflows running in the application, including authentication and, where applicable, real-time features. The demonstration should be rehearsed and have a fallback, such as a recorded run, in case of network problems.
4. Quality and security	Testing strategy and results, CI pipeline, security controls and remaining risks.
5. Process	How the team organized the work: sprints, board, branching, and reviews. Individual contributions.
6. Reflection	Challenges, decisions that changed, known limitations, and possible future work.
7. Questions	Answering technical questions about the code and design.

17. Official Technical References

Students should consult official documentation first and avoid outdated tutorials and unofficial claims. Version numbers in documentation change, so each student checks that the documentation matches the installed version.

Topic	Resource	URL
Web platform (HTML, CSS, accessibility)	MDN Web Docs	https://developer.mozilla.org/
JavaScript	MDN JavaScript documentation	https://developer.mozilla.org/en-US/docs/Web/JavaScript
HTML validation	W3C Markup Validation Service	https://validator.w3.org/
React	React documentation	https://react.dev/
React Router	React Router documentation	https://reactrouter.com/
Vue.js	Vue.js documentation	https://vuejs.org/guide/introduction.html

Topic	Resource	URL
Vue Router	Vue Router documentation	https://router.vuejs.org/
Vite	Vite documentation	https://vite.dev/guide/
Bootstrap	Bootstrap documentation	https://getbootstrap.com/docs/
Tailwind CSS	Tailwind CSS documentation	https://tailwindcss.com/docs
Node.js	Node.js documentation and learning resources	https://nodejs.org/docs/latest/api/ and https://nodejs.org/en/learn
Node.js releases	Node.js release schedule	https://nodejs.org/en/about/previous-releases
Express.js	Express documentation	https://expressjs.com/
GraphQL	GraphQL learning resources	https://graphql.org/learn/
Apollo Server	Apollo GraphQL documentation	https://www.apollographql.com/docs/apollo-server/
Socket.IO	Socket.IO documentation	https://socket.io/docs/v4/
PostgreSQL	PostgreSQL documentation	https://www.postgresql.org/docs/
Sequelize	Sequelize documentation (version 6)	https://sequelize.org/docs/v6/
MongoDB	MongoDB documentation	https://www.mongodb.com/docs/
MongoDB Atlas	Atlas documentation	https://www.mongodb.com/docs/atlas/
Docker	Docker documentation	https://docs.docker.com/
Docker Compose	Compose documentation	https://docs.docker.com/compose/
Git	Git documentation	https://git-scm.com/doc
GitHub	GitHub documentation	https://docs.github.com/
GitHub Actions	GitHub Actions documentation	https://docs.github.com/en/actions
Security	OWASP Top 10	https://owasp.org/www-project-top-ten/
Security	OWASP Cheat Sheet Series	https://cheatsheetseries.owasp.org/

Topic	Resource	URL
JSON Web Tokens	RFC 7519	https://www.rfc-editor.org/rfc/rfc7519
Commit conventions (Recommended Practice)	Conventional Commits specification	https://www.conventionalcommits.org/
Testing (Optional Technology)	Node.js test runner	https://nodejs.org/api/test.html
API specification (Optional Technology)	OpenAPI Specification	https://spec.openapis.org/oas/latest.html

The OWASP project page links to the current edition of the Top 10. Links were selected as the official documentation sites of each technology and should be verified when the document is reviewed, because sites may reorganize their addresses.

18. Appendices and Templates

Appendix A. Project proposal template

- Project title and team members with roles.
- Overview (maximum one paragraph).
- Problem statement and justification.
- General objective and specific objectives.
- Target users and stakeholders.
- Initial list of main features (epics).
- Proposed technologies and the reason for each, referring to Section 7.2.
- Main risks and assumptions.

Appendix B. Requirements specification template

ID	Type	Requirement statement	Priority	Acceptance criterion	Source or user
FR-01	Functional				
NFR-01	Non-functional				
BR-01	Business rule				

Appendix C. Architecture Decision Record template

```
# ADR-000: Title of the decision

Status: Proposed | Accepted | Superseded by ADR-000
Date: YYYY-MM-DD

## Context
What problem or requirement leads to this decision? What constraints
apply?

## Options considered
1. Option A: advantages and disadvantages.
2. Option B: advantages and disadvantages.

## Decision
The option chosen and the reasons, referring to requirements and team
skills.

## Consequences
Benefits, costs, risks, and what must be done as a result.
```

Appendix D. Issue template

```
## User story
As a [type of user], I want [functionality] so that [expected
benefit].

## Acceptance criteria
- [ ] Given ..., when ..., then ...
- [ ] ...

## Technical notes
Affected components, API changes, database changes.

## Definition of Done
- [ ] Tests written and passing
- [ ] Reviewed and merged
- [ ] Documentation updated
```

Appendix E. Pull request template

```
## Summary
What does this pull request change and why?

Closes #
```

```
## How it was tested
Commands run, tests added, manual checks.

## Checklist
- [ ] Follows the code style and the agreed architecture
- [ ] Tests added or updated and passing
- [ ] No secrets or credentials in the changes
- [ ] Input validation and access control considered
- [ ] Documentation updated (README, API, diagrams)
```

Appendix F. Security register template

ID	Risk description	OWASP category	Severity	Control	Status	Evidence
SEC-01						

Appendix G. Pre-submission technical checklist

Area	Check	Done
Repository	No secrets in the code or history; .env is ignored; .env.example is complete.	
Setup	A new machine can run the system by following the README only.	
Backend	Validation on every endpoint; centralized error handling; correct status codes; protected routes enforce roles.	
Database	Migrations run from an empty database; constraints and relationships match the ERD.	
Frontend	Works on small and large screens; forms have labels; errors are shown to the user; logout works.	
Testing	The automated suite passes locally and in CI; critical paths and error cases are covered.	

Area	Check	Done
Security	Security register is current; npm audit reviewed; CORS and rate limiting configured.	
DevOps	Docker Compose starts all services; CI passes on the main branch; branch protection is enabled.	
Documentation	Diagrams match the implementation; API documentation matches the endpoints.	
Presentation	Demonstration rehearsed with test data and a fallback.	

Appendix H. Items requiring instructor confirmation

This document does not invent institutional requirements. The following items are proposals or open decisions that must be confirmed or defined by the instructor.

No.	Item	Section
1	The wording of the general and specific objectives.	3.1, 3.2
2	Which statements are mandatory requirements for all projects.	3.3
3	Testing tools allowed or required (Node.js test runner, Jest, Vitest, Supertest, Playwright).	4.6, 11
4	Sprint length and the sprint ceremonies expected.	8.1
5	Changes to the team's Definition of Done when used for evaluation.	8.3
6	Whether a single repository is required or separate repositories are allowed.	9.2

No.	Item	Section
7	Ownership and licensing rules for Capstone work.	9.4
8	Duration of each roadmap phase and project milestones.	10.1
9	Acceptable deployment targets and whether deployment is mandatory.	13.2, 15.1
10	Which deliverables are mandatory and their due dates.	15.1
11	Evaluation weightings, rubric levels, and passing criteria.	15.2
12	Presentation duration, format, and participation rules.	16
13	Team size, individual project policy, and whether MongoDB, GraphQL, or Socket.IO use is required in any form.	3.3, 7.1

Appendix I. Glossary

Term	Meaning
Middleware	A function in Express that runs between receiving a request and sending a response, for example authentication or validation.
Migration	A versioned script that changes the database schema.
Seeder	A script that inserts initial or test data.
Resolver	A GraphQL function that produces the value of a field.
JWT	JSON Web Token, a signed token that carries claims about a user.
Room	A Socket.IO channel to which sockets are added so that events can be sent to a group.

Term	Meaning
Container	A running instance of a Docker image, isolated from other processes.
Volume	Storage managed by Docker that persists beyond the life of a container.
Pipeline	An automated sequence of steps, such as install, test, and build, that runs on a trigger.
Technical debt	The accumulated cost of shortcuts in design or code that make later changes harder.